

Zero Boundary Algebra 1.0

A Typed State Calculus for Directional Boundaries, Mirror Symmetry, and Tamper-Evident Provenance

Shafaet Brady Hussain

Independent researcher; creator of Zero Boundary Algebra and architect of ZMath

Nottingham, United Kingdom

Contact: shaf@talktoai.org

ORCID: not asserted

Version 1.0.0 - 18 August 2026

Abstract

This paper defines Zero Boundary Algebra (ZBA) as a typed state calculus for representing entry into a boundary, an at-boundary reset, emergence from a boundary, polarity reversal, recursive verification, and provenance-preserving composition. The motivating notation uses the symbols -0 , 0 , and $+0$ for three operationally distinct boundary phases; 3 and 6 as human-readable positive and negative polarity labels; $9/0$ as completion or reset; and 8 as a mnemonic for an involutive mirror because $8 \equiv -1 \pmod{9}$. These symbols are labels and operators in a transition system, not claims that ordinary real arithmetic has been replaced.

The formal object is a many-sorted algebra $Z = (S, 0, A)$ whose states bind an asset identifier, polarity, boundary phase, recursion depth, policy, lineage, and cryptographic commitment. We specify typing rules, transition preconditions, an involutive mirror, an idempotent reset projection, an append-only evidence relation, and a distinction between exact state equality and audit equivalence. We prove core structural properties, identify the circumstances under which mirror and reset commute, and show how non-commutativity may encode history rather than contradiction. A reference record format, conformance levels, threat model, test vectors, and falsifiable evaluation programme are provided.

ZBA is intentionally positioned as an algebraic control and provenance framework under formal development. It does not itself encrypt data, establish cryptographic strength, provide post-quantum security, create quantum key distribution, prove legal ownership, or replace established provenance standards. Its practical role is to supply a compact, machine-checkable vocabulary around conventional authenticated encryption, signatures, hash chains, append-only logs, and interoperable provenance models.

Keywords: algebraic specification; labelled transition system; provenance; tamper evidence; directional zero; signed zero; state machine; authenticated metadata; audit log; ZMath.

1. Contribution and scope

Zero Boundary Algebra began as a symbolic vocabulary distributed across research notes and implementations. This paper turns that vocabulary into one testable object. Its principal contributions are:

1. a typed state space separating polarity, boundary phase, and evidence status;
2. explicit mirror, enter, seal/reset, emerge, verify, compose, and recurse operators;
3. axioms and derived propositions with declared limits;
4. a cryptographic binding profile for state metadata;
5. a mapping to W3C PROV concepts rather than a competing provenance ontology;
6. conformance levels that separate notation, transition validation, and cryptographic evidence;
7. a reproducible evaluation plan capable of rejecting the claim that ZBA improves audit work.

The paper makes no claim that the numerals 3, 6, 8, or 9 possess supernatural, physical, or cryptographic power. Their role is mnemonic. Security arises from reviewed cryptographic primitives and correct implementations. Audit value, if any, arises from explicit state semantics, transition constraints, and evidence binding.

2. Design requirements

The calculus is designed around eight requirements.

R1 - Direction matters. Arrival at a boundary and departure from it must be distinguishable even when a scalar projection maps both to zero.

R2 - State and evidence differ. A claimed state is not a verified state. Evidence status is represented separately.

R3 - Operations are typed. Ill-formed transitions fail closed instead of silently coercing values.

R4 - History is explicit. When an operation changes lineage, exact equality and audit equivalence are not conflated.

R5 - Metadata is bound. Security-relevant labels are authenticated or signed; decorative labels alone have no integrity force.

R6 - Conventional standards remain authoritative. ZBA is a domain-specific control layer, not a new cipher, hash, signature scheme, or provenance interchange standard.

R7 - Claims are falsifiable. Human and machine evaluations must compare ZBA against ordinary labelled-state baselines.

R8 - Private implementation is unnecessary for public verification. A verifier can validate a public record without private keys, plaintext, or proprietary application code, provided the relevant public keys, commitments, and policies are available.

3. Mathematical preliminaries

3.1 Sorts

Let the following carrier sets be non-empty:

- A: asset identifiers;
- $P = \{-, 0, +\}$: polarity labels;
- $B = \{-0, 0, +0\}$: boundary phases, read as entering, at-boundary, and emerging;

- N: non-negative recursion depths;
- Q: policy identifiers;
- H: fixed-length cryptographic commitments;
- L: finite event lineages;
- $E = \{\text{claimed}, \text{checked}, \text{sealed}, \text{rejected}\}$: evidence status;
- T: monotonically ordered logical times or sequence numbers.

A ZBA state is the tuple

$$s = (a, p, b, r, q, l, h, e) \text{ in } S = A \times P \times B \times N \times Q \times L \times H \times E.$$

The projections are written $\text{asset}(s)$, $\text{pol}(s)$, $\text{phase}(s)$, $\text{depth}(s)$, $\text{policy}(s)$, $\text{lineage}(s)$, $\text{commit}(s)$, and $\text{evidence}(s)$.

3.2 Directional zero is a tagged control value

The three values in B are not three real numbers. They are tagged control states. Define an erasure map

$$\text{epsilon} : B \rightarrow \{0\} \text{ with } \text{epsilon}(-0) = \text{epsilon}(0) = \text{epsilon}(+0) = 0.$$

Thus the tags are distinct in the control domain while sharing the same erased scalar value:

$$-0 \neq 0 \neq +0 \text{ in } B, \text{ but } \text{epsilon}(-0) = \text{epsilon}(0) = \text{epsilon}(+0).$$

This is analogous only in spirit to signed-zero representations in floating-point systems. ZBA attaches workflow meaning that IEEE floating-point arithmetic does not supply. Implementations must not serialize phase tags as bare JSON numbers: common canonicalization rules can collapse negative zero to zero. A conforming record serializes them as strings: "entering", "boundary", and "emerging".

3.3 Mnemonic numerals

Define a display map μ :

- $\mu(+) = 3$;
- $\mu(-) = 6$;
- $\mu(0) = 9/0$;
- $\mu(\text{mirror}) = 8$.

The mirror mnemonic follows $8 \equiv -1 \pmod{9}$, so multiplication by 8 reverses non-zero residues modulo 9 and is involutive because $8^2 \equiv 1 \pmod{9}$. This modular fact motivates the notation but does not by itself prove properties of a software system. Those properties follow only from the definitions below.

4. Signature and operators

Let $\mathcal{O}$ contain the following partial or total operations.

4.1 Polarity mirror

Define $\text{negP}(+) = -$, $\text{negP}(-) = +$, and $\text{negP}(0) = 0$.

The structural mirror $M : S \rightarrow S$ is

$$M(a, p, b, r, q, l, h, e) = (a, \text{negP}(p), b, r, q, l, h, e).$$

The evidence-preserving implementation operator M^* may additionally append an event and recompute the commitment. Accordingly, M models the algebraic effect and M^* models an audited execution.

4.2 Boundary transitions

For an operation identifier op , actor u , and logical time t , define partial transitions:

- $Enter_op(s)$ is defined when $phase(s) = +0$ or the asset is newly created; its result has phase -0 .
- $Seal_op(s)$ is defined when $phase(s) = -0$; its result has phase 0 and evidence at least checked.
- $Emerge_op(s)$ is defined when $phase(s) = 0$ and the policy's release predicate holds; its result has phase $+0$.
- $Reject_op(s)$ is defined for any unaccepted verification; its result has evidence rejected and cannot emerge without a new authorised recovery event.

The ordinary successful cycle is therefore

$+0 \rightarrow -0 \rightarrow 0 \rightarrow +0$.

The transition labels, not the phase tags alone, state what occurred: create, encrypt, rekey, export, recover, verify, revoke, or another policy-defined operation.

4.3 Reset projection

The structural reset $R : S \rightarrow S$ projects any permitted state to its at-boundary form:

$$R(a, p, b, r, q, l, h, e) = (a, 0, 0, r, q, l, h, e'),$$

where e' is checked if validation succeeds and rejected otherwise. In an audited system, R^* appends a seal event and recomputes h .

4.4 Recursive verification

The recursion operator $V : S \rightarrow S$ increments the verification depth after a successful verification:

$$V(a, p, b, r, q, l, h, e) = (a, p, b, r+1, q, l, h, sealed).$$

This operator does not mean mathematical infinity. Recursion is a finite, observable repetition count. Systems may impose a maximum depth and must reject wraparound.

4.5 Composition

For transition operators f and g , composition $g \circ f$ is defined only when $f(s)$ is defined and satisfies the preconditions of g . A composition carries the concatenated event sequence. Parallel asset histories form a product state; they do not implicitly share keys, authority, or evidence.

4.6 Audit equivalence

Define exact equality = componentwise over S . Define audit equivalence $\sim_a$ relative to an observer authorised for asset a :

$s \sim_a t$ iff their asset, policy, semantic payload commitment, and externally visible control state are equal, and any lineage difference consists only of valid, non-destructive audit events permitted by policy.

Audit equivalence is not cryptographic equality. It permits two states to describe the same controlled asset after reversible checks while retaining different evidence histories.

5. Axioms

The algebra adopts the following axioms.

A1 - Sort preservation. Every defined operator returns a state in S ; parsers reject unknown mandatory fields, invalid tags, depth underflow/overflow, and malformed commitments.

A2 - Asset invariance. Structural control operators do not change $\text{asset}(s)$.

A3 - Mirror involution. $M(M(s)) = s$.

A4 - Reset idempotence. $R(R(s)) = R(s)$ for states whose validation outcome is stable.

A5 - Phase order. A successful controlled operation follows $+0 \rightarrow -0 \rightarrow 0 \rightarrow +0$; shortcuts require a separately named exception transition.

A6 - Evidence monotonicity. Within an uncompromised lineage, a record cannot silently move from rejected to sealed, nor delete an event. Recovery creates a new event and preserves the rejected predecessor.

A7 - Commitment recurrence. For canonical event bytes $C(E_n)$, previous commitment $h_{(n-1)}$, state projection $C(S_n)$, and domain separator D ,

$$h_n = H(D \parallel h_{(n-1)} \parallel C(E_n) \parallel C(S_n)).$$

A8 - Authorisation. A transition is valid only if the actor, policy version, operation, and preconditions are authorised and the relevant signature or MAC verifies.

A9 - Nonce discipline. If an AEAD profile is used, its nonce uniqueness and key-management requirements are independent invariants; ZBA labels cannot repair nonce reuse.

A10 - Verifier determinism. Given the same canonical record, policy, trust anchors, and validation time, conforming verifiers return the same result.

6. Propositions and proofs

Proposition 1 - Mirror is an involution

For every s in S , $M^2(s) = s$.

Proof. M changes only the polarity component using negP . The map swaps $+$ and $-$ and fixes 0 , so $\text{negP}(\text{negP}(p)) = p$. Every other component is unchanged. Therefore applying M twice returns every component of s . QED.

For the audited implementation M^* , exact equality usually fails because two events have been appended. The appropriate invariant is $M^*(M^*(s)) \sim_a s$, provided both mirror events verify and the policy treats them as semantically reversible.

Proposition 2 - Structural reset is idempotent

For every state with stable validation outcome, $R^2(s) = R(s)$.

Proof. One application sets polarity and phase to their boundary values and chooses a deterministic evidence result. A second application makes the same assignments. All other structural components are fixed. QED.

The audited operator R^* need not be exactly idempotent because repeated sealing attempts may append distinct observations. Conforming systems either reject a redundant seal or classify it as a no-op observation.

Proposition 3 - Mirror and reset commute structurally

For the definitions above, $R(M(s)) = M(R(s))$.

Proof. R sets polarity to $\emptyset$, which M fixes. Both sides set the phase to $\emptyset$ and preserve the other structural components. QED.

This proposition is deliberately narrow. Audited implementations may not commute: $R^* \circ M^*$ and $M^* \circ R^*$ append different ordered events and therefore produce different commitments. They can be semantically related while remaining cryptographically distinct. This ordered difference is useful because it records whether reversal occurred before or after sealing.

Proposition 4 - Single-event tampering propagates

Assume collision resistance of H , unambiguous canonicalization, and a trusted anchor for h_0 . If any committed event or state component at index i is modified without recomputing and reauthorising all later records, verification fails at i or a later anchor.

Proof sketch. By A7, h_i depends on the previous commitment and canonical bytes at i ; every later h_j depends recursively on h_i . A changed input changes the digest except with a collision or preimage-style failure. If an attacker recomputes later hashes, signatures or external anchors no longer match unless the attacker also controls the corresponding authority. QED under the stated assumptions.

Proposition 5 - A label without binding supplies no tamper evidence

If phase, polarity, or policy is excluded from the authenticated data, signature payload, and hash-chain input, an attacker capable of editing the record may relabel it without invalidating cryptographic checks.

Proof. Verification is a function only of bound inputs. Changing an unbound field does not change that function's input, so the check result is unchanged. QED.

This proposition is the central engineering constraint: symbolic provenance becomes security-relevant only when it is cryptographically bound.

7. Canonical record and cryptographic binding profile

7.1 Record fields

A minimum public record contains:

- spec: fixed identifier and version;
- asset_id: opaque stable identifier;
- sequence: unsigned monotonically increasing integer;
- operation: policy-defined transition label;
- polarity: positive, neutral, or negative;
- phase: entering, boundary, or emerging;
- recursion_depth: unsigned integer;
- policy_id and policy_version;
- previous_commitment;
- payload_commitment;
- event_time or a declared logical clock;

- actor_id or privacy-preserving actor reference;
- evidence_status;
- commitment_algorithm and commitment;
- optional signature suite, public-key reference, and signature;
- optional external anchor reference.

7.2 Canonicalization

Interoperable hashing requires one byte representation. The reference profile uses UTF-8 JSON canonicalized according to RFC 8785, but phase and polarity values are strings. Unknown optional fields may be preserved; unknown critical fields cause rejection. Domain-separation strings distinguish event commitments, payload commitments, and signature inputs.

7.3 Authenticated encryption

Where the record accompanies an encrypted payload, the ZBA control projection should be included as authenticated additional data (AAD) or in a signed manifest. NIST SP 800-38D defines GCM as authenticated encryption with associated data: AAD is authenticated but not encrypted. A ZBA implementation therefore may keep non-sensitive control metadata visible while making unauthorised changes detectable. Visibility can leak workflow information, so sensitive metadata should instead be encrypted or tokenised.

Two layers of AES-GCM do not constitute AES-512 and do not automatically double security. If a system uses two layers, the defensible rationale is factor separation, compartmentalisation, or defence in depth, subject to independent keys, correct nonces, and a reviewed composition.

7.4 Public-key and post-quantum boundary

Classical ephemeral ECDH is not post-quantum secure against a sufficiently capable cryptographic quantum computer. A future hybrid profile may combine a classical agreement with a standardised post-quantum KEM such as ML-KEM, defined in NIST FIPS 203. Merely recording a quantum-computer job identifier does not provide post-quantum confidentiality.

7.5 External anchors

An external execution record can bind a provider job ID, circuit fingerprint, backend, status, result reference, and the ZBA commitment. This is an evidence anchor, not a quantum encryption primitive and not QKD. Its value depends on authenticity, availability, uniqueness, binding, and the independence of the external record. Append-only Merkle logs or trusted timestamps are alternative anchor mechanisms.

8. Mapping to established provenance concepts

ZBA is a domain-specific state vocabulary. W3C PROV remains the interoperability layer. A recommended mapping is:

ZBA concept	W3C PROV concept	Qualification
protected asset/version	prov:Entity	Each materially changed version is a new entity.
encrypt, rekey, verify, export	prov:Activity	Transition event with start/end or logical sequence.
operator, device, service	prov:Agent	Responsibility and attribution are policy-dependent.

ZBA concept	W3C PROV concept	Qualification
input to operation	prov:used	The activity used the preceding entity.
output version	prov:wasGeneratedBy	The successor entity was generated by the activity.
lineage edge	prov:wasDerivedFrom	Does not by itself prove integrity.
author attribution	prov:wasAttributedTo	Attribution is not identical to legal ownership.
policy	prov:Plan	The plan describes permitted transitions.
ZBA evidence bundle	prov:Bundle	A named set of provenance descriptions.

ZBA adds phase, polarity, transition guards, and commitment recurrence. It should export rather than replace PROV-compatible statements.

9. Threat model

9.1 Protected properties

The model seeks to support integrity of transition metadata, detection of broken lineage, explicit policy/version binding, reproducible verification, and separation of claimed from checked evidence.

9.2 Adversaries

Consider an adversary who may read or alter stored records, reorder or delete events, replay an old valid record, relabel a state, substitute a payload, exploit parser differences, compromise an endpoint, steal an application credential, or present an unrelated external job as an anchor.

9.3 Required mitigations

- hash chaining or an append-only authenticated structure;
- signatures or MACs with explicit trust anchors;
- sequence and freshness policy;
- canonical serialization and critical-field handling;
- binding of asset, operation, phase, polarity, policy, payload commitment, and predecessor;
- domain separation;
- rollback detection using an independent checkpoint or witness;
- endpoint hardening and key lifecycle controls;
- privacy review for visible metadata;
- recovery procedures that append rather than erase failures.

9.4 Out of scope

ZBA alone does not prevent endpoint compromise, key theft, traffic analysis, denial of service, coercion, malicious authorised actors, cryptographic implementation bugs, long-term algorithm failure, or false source data entering an otherwise intact log. Provenance can show what a system recorded; it cannot guarantee that the recorded event was truthful without trustworthy sensing and authority.

10. Conformance levels

Level 0 - Notation. Uses the vocabulary but makes no machine-verifiable integrity claim.

Level 1 - Typed transitions. Validates the state schema, phase order, guards, policy versions, and deterministic error results.

Level 2 - Bound evidence. Implements canonical commitments over all critical fields and verifies predecessor continuity.

Level 3 - Authenticated evidence. Adds signatures or MACs, trust-anchor handling, algorithm identifiers, key status, and replay/rollback controls.

Level 4 - Witnessed evidence. Adds an independent trusted timestamp, transparency log, notarisation, or external execution anchor with a documented binding protocol.

A product must state its level and profile. It must not call a Level 0 decorative label tamper-evident.

11. Reference transition policy

The reference policy is fail-closed:

Current phase	Requested operation	Preconditions	Next phase	Failure
emerging/new	enter operation	actor and operation authorised	entering	reject event
entering	seal/reset	payload and policy checks pass	boundary	rejected boundary
boundary	emerge/release	release predicate and signature pass	emerging	remain boundary/rejected
any	verify	chain, signatures, policy and payload commitment pass	unchanged; depth + 1	evidence rejected
any	recover	recovery authority and predecessor preserved	entering	reject event

An emergency transition may exist, but it must be a named policy operation with a reason code, elevated authorisation, and subsequent review. It must never be disguised as a normal cycle.

12. Test vectors and verifier obligations

The accompanying reference artifacts define deterministic positive and negative cases. At minimum, a verifier must test:

1. a valid create-enter-seal-emerge sequence;
2. mirror twice, demonstrating structural equality and distinct audited commitments;
3. reset twice, demonstrating structural idempotence or a declared no-op event;
4. mutation of phase, polarity, operation, policy, predecessor, payload commitment, and depth;
5. deletion, insertion, duplication, and reordering of events;
6. wrong signature, unknown key, revoked key, and unsupported critical field;
7. rollback to an earlier valid prefix when an external checkpoint exists;
8. JSON -0, duplicate keys, invalid Unicode, oversized integers, and non-canonical encodings;
9. incorrect AEAD nonce reuse detection at the surrounding cryptographic layer;
10. external anchor mismatch between job reference and committed record.

The verifier must return a structured result with the first failing sequence, failure class, critical field, and policy version. Boolean-only results are insufficient for audit use.

13. Evaluation programme

13.1 Research questions

- RQ1: Does ZBA reduce the time reviewers need to reconstruct an asset's transition history?
- RQ2: Does it improve detection of contradictory, missing, reordered, or unauthorised states?
- RQ3: Does the mnemonic notation add value beyond an ordinary finite-state-machine vocabulary?
- RQ4: Can independent implementations produce identical verification results?
- RQ5: What privacy and cognitive costs arise from the additional metadata?

13.2 Baselines

Compare three conditions:

- B0: ordinary timestamped application logs;
- B1: a conventional typed provenance state machine with descriptive labels;
- B2: the same state machine with ZBA directional-phase and mirror vocabulary.

B1 is essential. Comparing only against unstructured logs would not isolate whether ZBA's distinctive notation adds value.

13.3 Tasks and measures

Construct blinded record sets with valid histories and seeded defects. Participants answer lineage, authority, contradiction, and recovery questions. Primary outcomes are defect-detection F1, time to correct reconstruction, inter-rater agreement, and false assurance rate. Secondary outcomes are training time, perceived workload, terminology recall, verifier interoperability, record-size overhead, and privacy leakage identified by reviewers.

13.4 Hypotheses and rejection criteria

- H1: B2 improves defect-detection F1 over B0.
- H2: B2 is non-inferior to B1 on accuracy and faster on boundary reconstruction.
- H3: independent Level 2 verifiers agree on every normative vector.

If B2 does not outperform or reach non-inferiority against B1, the distinctive ZBA notation has not demonstrated operational value even if the underlying state machine remains sound. If independent verifiers disagree, the specification is ambiguous and must be revised. If users systematically confuse evidence status with cryptographic assurance, the vocabulary must be redesigned.

13.5 Security evaluation

Use property-based testing for phase transitions and mirror/reset laws; mutation testing for every critical field; parser differential testing; fuzzing of canonical records; signature and trust-store negative tests; AEAD misuse checks; rollback simulations; and an independent threat-model review. Public results must distinguish specification conformance from a full product security audit.

14. Implementation evidence and boundaries

The wider ZMath ecosystem provides motivating implementations: browser-side protected artifacts, encrypted messaging workflows, portable containers, provenance records, and optional external execution evidence. These show that the vocabulary has been applied across multiple software surfaces. They do not constitute independent validation of the algebra, cryptographic review, quantum advantage, or formal verification.

For Zmail-style dual-input protection, the public claim should remain precise: separate passphrase and visual-pattern inputs may derive independent key material using separate salts and standard KDFs, followed by authenticated encryption. The security contribution is factor separation and implementation discipline, not a multiplication of AES key length. Ordinary email metadata and unsupported delivery paths can remain outside end-to-end protection and must be disclosed.

For QuantumEncryption1-style evidence, provider job IDs and circuit fingerprints can serve as external provenance references when cryptographically bound. They must not be described as secretly generating encryption keys unless a documented key protocol actually does so, and they are not evidence of QKD.

15. Limitations

This is a first public formalisation by the framework's creator, not peer review or independent validation. The axioms are a design proposal. Some choices, especially audit equivalence and policy semantics, require domain-specific refinement. The paper supplies proof sketches for structural properties, not a mechanised proof assistant development. No user study is reported. No claim of mathematical novelty relative to all prior labelled transition systems, process algebras, event sourcing, temporal logic, or provenance systems is made. The originality claim is narrower: the particular directional-boundary, polarity, mirror, and recursive-evidence synthesis named Zero Boundary Algebra, together with this specification.

Legal provenance is also limited. Hashes, signatures, timestamps, and public releases may strengthen an evidence trail, but they do not automatically establish patentability, copyright ownership, trade-secret status, contractual rights, authorship priority in every jurisdiction, or institutional endorsement.

16. Research roadmap

1. Publish the schema, vectors, verifier, and immutable release digest.
2. Invite mathematical, security, provenance, and human-factors review.
3. Mechanise the core transition system in TLA+, Alloy, Lean, or Coq.
4. Implement W3C PROV export and validate round-trip semantics.
5. Run the B0/B1/B2 blinded evaluation with preregistered metrics.
6. Commission an independent cryptographic design review of any ZMath profile claiming Level 3 or 4.
7. Define a hybrid post-quantum profile only after threat, interoperability, downgrade, and key-lifecycle analysis.
8. Revise the algebra based on counterexamples and verifier divergence.

17. Conclusion

Zero Boundary Algebra is most defensibly understood as a typed algebraic state calculus for directional reset phases, polarity mirror operations, recursive verification, and cryptographically bound provenance. Its distinctive symbols are useful only insofar as they make system state clearer, constrain transitions, and improve verification. The formal mirror is involutive; the structural reset is idempotent; the two commute in the structural algebra while their audited executions can preserve meaningful order. These properties give the framework a precise core without pretending that notation creates cryptographic strength.

The next scientific question is empirical: whether reviewers and systems using ZBA detect defects more reliably than those using ordinary provenance state machines. By publishing the specification, boundaries, negative tests, and rejection criteria together, the framework becomes open to the kind of criticism that can turn an originating idea into durable research.

References

1. Hussain, S. B. (2026). *Creating a New Mathematical System 0,-0,+0*. Research Forum Online. <https://research.talktoai.org/research-papers/creating-a-new-mathematical-system/>
2. Hussain, S. B. (2026). *Zero Boundary Algebra as a Provenance Workflow for AI Research Notes*. ResearchForumOnline Research repository.
3. W3C. (2013). *PROV-O: The PROV Ontology*. W3C Recommendation. <https://www.w3.org/TR/prov-o/>
4. Moreau, L., & Missier, P. (eds.). (2013). *PROV-DM: The PROV Data Model*. W3C Recommendation. <https://www.w3.org/TR/prov-dm/>
5. Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D. <https://doi.org/10.6028/NIST.SP.800-38D>
6. NIST. (2024). *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203. <https://doi.org/10.6028/NIST.FIPS.203>
7. Rundgren, A., Jordan, B., & Erdtman, S. (2020). *JSON Canonicalization Scheme (JCS)*. RFC 8785. <https://doi.org/10.17487/RFC8785>
8. Laurie, B., Langley, A., & Kasper, E. (2013). *Certificate Transparency*. RFC 6962. <https://doi.org/10.17487/RFC6962>
9. NIST. *Hash chain*. Computer Security Resource Center Glossary. https://csrc.nist.gov/glossary/term/hash_chain
10. IEEE. (2019). *IEEE Standard for Floating-Point Arithmetic*. IEEE 754-2019. <https://standards.ieee.org/ieee/754/6210/>
11. QuantumEncryption1. (2026). *Zero Boundary Algebra for encrypted provenance and control*. <https://quantumencryption1.com/zero-boundary-algebra/>
12. QuantumEncryption1. (2026). *Technology architecture*. <https://quantumencryption1.com/technology/>

Appendix A - Normative invariants

1. $\text{asset}(s') = \text{asset}(s)$ for every structural control transition.
2. $M(M(s)) = s$.
3. $R(R(s)) = R(s)$ under stable validation.
4. $R(M(s)) = M(R(s))$ structurally.
5. Successful normal phases follow emerging -> entering -> boundary -> emerging.
6. Every non-genesis record commits to exactly one predecessor within a single asset lineage.
7. Every critical field is included in the canonical commitment and signature/MAC input.
8. A rejected event is never silently overwritten.
9. Phase values are tagged strings, not signed numeric zero.
10. Verification is deterministic for fixed inputs and trust state.

Appendix B - Claim boundary

Statement	Status
ZBA defines a typed state calculus	Defined in this specification
Structural mirror is involutive	Proven from the definition
Structural reset is idempotent	Proven under stable validation
Hash-linked, authenticated records can expose mutation	Conditional on cryptographic and implementation assumptions
ZBA improves reviewer performance	Hypothesis; not yet demonstrated
ZMath products implement every conformance level	Not asserted
Two AES-GCM layers equal AES-512	Rejected
ECDH is post-quantum secure	Rejected
Quantum execution evidence is QKD or quantum encryption	Rejected
Publication proves legal ownership or institutional endorsement	Rejected